

使用 ADK 建構 AI 代理：加入工具

來源：<https://codelabs.developers.google.com/devsite/codelabs/build-agents-with-adk-empowering-with-tools?hl=zh-tw>

步驟數：8

本程式碼研究室將引導您逐步操作，為基本 ADK 代理程式提供各種工具。首先，您要建立自訂 FunctionTool，擷取即時匯率資料。接下來，您將整合 Google 搜尋做為專用子代理程式，瞭解核心架構模式。最後，您將從 LangChain 新增第三方維基百科工具，將簡單的聊天機器人變成精密的旅遊助理，可智慧協調多項功能，處理複雜的使用者要求。

目錄

1. [1. 事前準備](#)
2. [2. 簡介](#)
3. [3. 開始使用：基礎代理程式](#)
4. [4. 建立貨幣兌換自訂工具](#)
5. [5. 與內建 Google 搜尋工具整合](#)
6. [6. 善用 LangChain 的維基百科工具](#)
7. [7. 清除 \(選用\)](#)
8. [8. 結論](#)

1. 事前準備

請詳閱：

本教學課程需要具備有效帳單帳戶的 Google Cloud 專案。

現場研討會參加者

- 請按照老師提供的具體設定和帳單說明操作。

自學使用者

- 如要兌換試用期，請按照下列步驟操作：
 - 在瀏覽器中開啟無痕視窗。
 - 前往[這個兌換入口網站](#)。
 - 使用個人 **Gmail** 帳戶登入。
 - 按照入口網站的逐步指示操作。

歡迎來到「使用 ADK 建構 AI 代理」系列文章的第二部分！在本實作程式碼研究室中，您將為基本 AI 代理程式提供各種工具。

如要開始使用，本指南提供兩條路徑：一條適用於從「[使用 ADK 建構 AI 代理：基礎知識](#)」程式碼實驗室繼續學習的學員，另一條則適用於從頭開始的學員。無論選擇哪種方式，都能確保您擁有必要的基礎代理程式碼。

完成本程式碼研究室後，您將為個人助理代理程式提供各種用途的工具，並在後續的系列課程中，將其擴充為複雜的多代理系統 (MAS)。

您也可以透過這個縮短網址存取程式碼研究室：goo.gle/adk-using-tools。

必要條件

- 瞭解[生成式 AI 概念](#)
- 具備 [Python 程式設計](#) 基本能力
- 完成[使用 ADK 建構 AI 代理：基礎知識](#)程式碼實驗室或類似課程

課程內容

- 建立自訂 Python 函式做為工具，賦予代理程式新技能。
- 使用 Google 搜尋等內建工具，將代理連結至即時資訊。
- 建立專用于代理來處理複雜工作，建構多工具代理。
- 整合 LangChain 等熱門 AI 架構的工具，快速擴充功能。

軟硬體需求

- 可正常運作的電腦和穩定的 Wi-Fi
- 瀏覽器 (例如 [Chrome](#))，用來存取 [Google Cloud 控制台](#)
- 好奇心和學習熱忱

2. 簡介

使用 ADK 建構的基本代理程式具有強大的 LLM 腦力，但也有其限制：無法存取訓練日期之後建立的資訊，也無法與外部服務互動。就像一位聰明又博學的助理被鎖在圖書館裡，沒有手機或網路。如要讓代理程式真正發揮效用，必須提供工具。

工具就像是提供 AI 助理存取外部世界的管道：計算機、網路瀏覽器，或是特定公司資料庫的存取權。在 ADK 中，工具是模組化程式碼，可讓代理執行特定動作，例如查詢即時資料或呼叫外部 API。使用工具可大幅擴展功能，不只能進行簡單的對話。

ADK 提供三類工具：

1. **函式工具**：您開發的自訂工具，可滿足應用程式的獨特需求，例如預先定義的函式和代理程式。
2. **內建工具**：框架提供的立即可用工具，用於處理一般作業，例如 Google 搜尋和程式碼執行。
3. **第三方工具**：熱門外部程式庫，例如 [Serper](#)，以及 LangChain 和 CrewAI 的工具。

如要進一步瞭解如何搭配使用 ADK 代理與各種工具，請參閱[官方說明文件](#)。在本程式碼研究室中，我們將新增工具，將簡單的代理程式轉換為功能強大的個人旅遊助理。我們馬上開始！

步驟 3 · 約 5 分鐘

3. 開始使用：基礎代理程式

如要為代理程式提供工具，您必須先有可使用的基本代理程式。請選擇最符合進度的路徑。

路徑 A：從基礎程式碼研究室繼續

如果您剛完成「[使用 ADK 建構 AI 代理：基礎知識](#)」程式碼研究室，就已準備就緒。您可以繼續在現有的 `ai-agents-adk` 專案目錄中工作。

路徑 B：從頭開始

如果您直接開始進行本程式碼研究室，請完成下列 4 個步驟，設定環境並建立必要的起始代理程式。

1. [設定 Google Cloud 服務](#)
2. [建立 Python 虛擬環境](#)
3. [建立代理](#)
4. [在開發 UI 上執行代理](#)

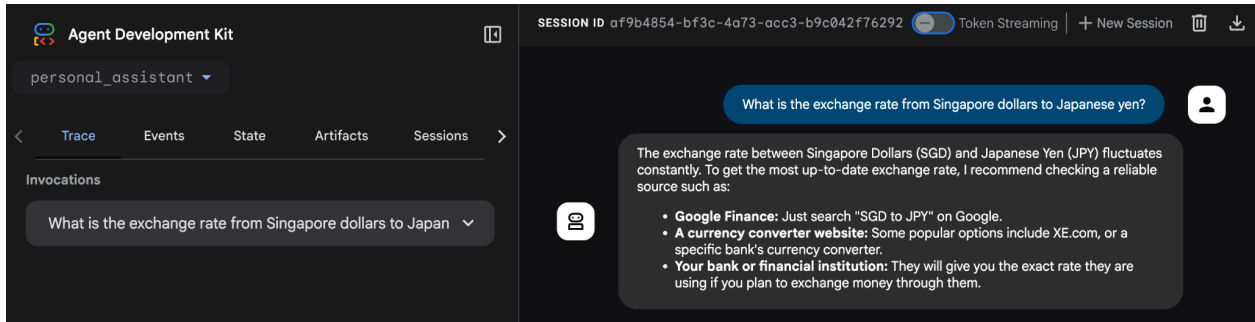
完成上述步驟後，即可開始學習之旅。

步驟 4 · 約 5 分鐘

4. 建立貨幣兌換自訂工具

此時，您應該已瞭解如何使用 ADK 建構簡單的 AI 代理，並在開發 UI 中執行。

假設您下個月要前往日本，需要查詢目前的匯率。詢問代理程式「新加坡元兌換日圓的匯率是多少？」



你會發現代理程式無法擷取即時匯率。這是因為代理程式目前無法存取網際網路，也無法連線至外部系統。即使代理程式回覆值，也很難信任該值，因為這很可能是幻覺。

為解決這個問題，我們將實作 Python 函式，透過 REST API 擷取匯率，並將其整合為代理的函式工具。

在終端機視窗中，使用鍵盤快速鍵 **Ctrl + C** 終止執行中的代理程序。

建立 `custom_functions.py` 檔案

注意：繼續操作前，請確認您位於正確的目錄 (`ai-agents-adk`)。您可以在終端機中執行 `pwd` (列印工作目錄) 指令，檢查目前位置。

在終端機中執行這項指令，即可在 `personal_assistant` 資料夾中建立名為 `custom_functions.py` 的 Python 檔案，並在程式碼編輯器中開啟該檔案。

```
cloudshell edit personal_assistant/custom_functions.py
```

現在的資料夾結構應如下所示：

```
ai-agents-adk/
├── personal_assistant/
│   ├── .env
│   ├── __init__.py
│   ├── agent.py
│   └── custom_functions.py
```

這個 `custom_functions.py` 檔案會包含負責從外部 API 擷取匯率資料的 Python 函式。複製下列程式碼並貼到檔案中：

```

import requests

# define a function to get exchange rate
def get_fx_rate(base: str, target: str):
    """
    Fetches the current exchange rate between two currencies.

    Args:
        base: The base currency (e.g., "SGD").
        target: The target currency (e.g., "JPY").

    Returns:
        The exchange rate information as a json response,
        or None if the rate could not be fetched.
    """
    base_url = "https://hexarate.paikama.co/api/rates/latest"
    api_url = f"{base_url}/{base}?target={target}"

    response = requests.get(api_url)
    if response.status_code == 200:
        return response.json()

```

注意：請仔細查看三引號 ("""...") 內的說明文字，這稱為「說明字串」。這對人類開發人員來說是不錯的做法，但對 AI 代理來說至關重要。

代理的基礎 LLM 會讀取函式名稱和這個 docstring，瞭解工具的作用、引數 (本例中為 `base` 和 `target`) 的意義，以及工具的使用時機。

清楚詳盡的 docstring 是代理正確使用工具的最重要因素。

現在請編輯 `agent.py` 檔案：匯入 `get_fx_rate` 函式，並指派為 `FunctionTool`。

更新 `agent.py` 檔案

複製這個程式碼區塊，並取代 `agent.py` 檔案的現有內容：

```

from google.adk.agents import Agent
from google.adk.tools import FunctionTool

from .custom_functions import get_fx_rate

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='A helpful assistant for user questions.',
    instruction='Answer user questions to the best of your knowledge',
    tools=[FunctionTool(get_fx_rate)]
)

```

附註：ADK 會將代理程式資料夾做為 Python 模組執行，因此需要相對匯入，才能從其他 Python 檔案匯入函式。如果工具數量不多，這種做法還算可行，但隨著工具數量增加，我們就需要建立工具套件。

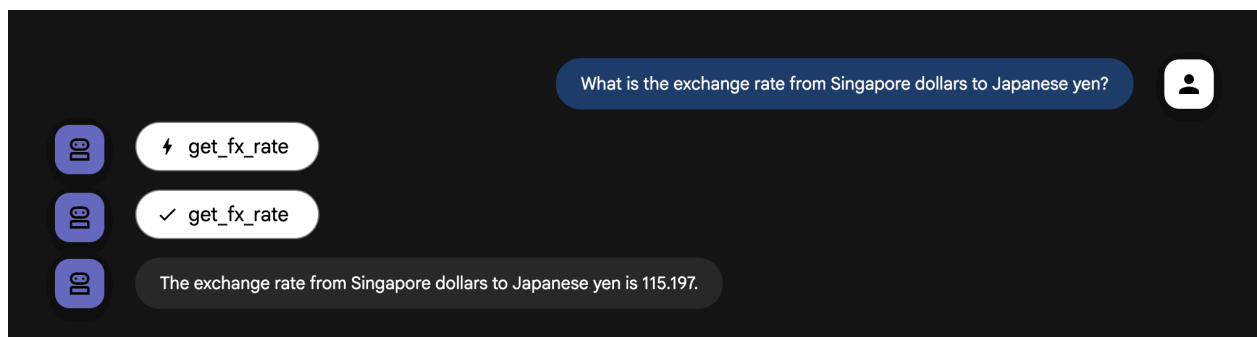
變更後，請輸入下列內容，再次啟動代理程式：

```
adk web --allow_origins "regex:https://.*\.cloudshell\.dev"
```

注意：在大多數情況下，執行 `adk web` 指令就已足夠，但如要確保從 Cloud Shell 終端機執行伺服器時能正常存取，請務必加上 `--allow_origins` 旗標。

代理程式啟動後，請再次提出相同問題：「新加坡元兌換日圓的匯率是多少？」

這次您應該會看到 `get_fx_rate` 工具提供的實際匯率。

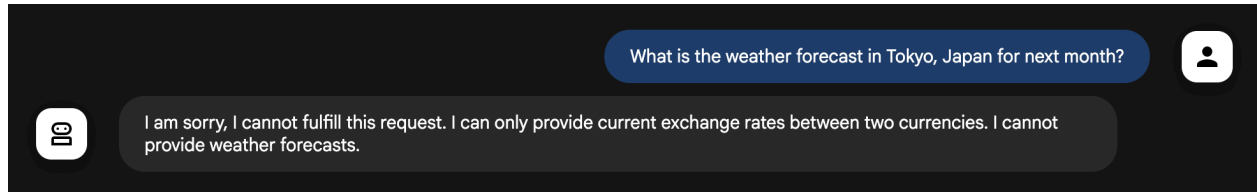


歡迎隨時提出任何貨幣兌換相關問題。

步驟 5 · 約 5 分鐘

5. 與內建 Google 搜尋工具整合

現在代理程式可以提供匯率，下一個工作是取得下個月的天氣預報。向代理提出這個問題：「What is the weather forecast in Tokyo, Japan for next month?」（日本東京下個月的天氣預報如何？）



如您所料，天氣預報需要即時資訊，但我們的智慧助理沒有這類資訊。雖然我們可以為每個需要即時資料的用途編寫新的 Python 函式，但新增越來越多的自訂工具，很快就會讓代理程式過於複雜，難以管理。

幸好，Agent Development Kit (ADK) 提供一系列**內建工具**，包括 Google 搜尋，可供直接使用，簡化代理與外部世界的互動方式。

如要為代理程式配備 Google 搜尋工具，您需要實作多代理程式模式。首先，您要建立專用代理，唯一的工作就是執行 Google 搜尋。接著，將這個新的 **Google 搜尋代理** 指派給主要 `personal_assistant` 做為工具。步驟如下：

建立 `custom_agents.py` 檔案

注意：請先確認您位於正確的目錄 (`ai-agents-adk`)，再繼續操作。在終端機中執行 `pwd` (列印工作目錄) 指令，確認目前位置。

在終端機中執行下列指令，即可在 `personal_assistant` 資料夾中建立名為 `custom_agents.py` 的 Python 檔案，並在程式碼編輯器中開啟該檔案：

```
cloudshell edit personal_assistant/custom_agents.py
```

現在的資料夾結構應如下所示：

```
ai-agents-adk/
├── personal_assistant/
│   ├── .env
│   ├── __init__.py
│   ├── agent.py
│   ├── custom_functions.py
│   └── custom_agents.py
```

這個 `custom_agents.py` 檔案會包含專屬 `google_search_agent` 的程式碼。將下列程式碼複製到 `custom_agents.py` 檔案中：

```

from google.adk.agents import Agent
from google.adk.tools import google_search

# Create an agent with google search tool as a search specialist
google_search_agent = Agent(
    model='gemini-2.5-flash',
    name='google_search_agent',
    description='A search agent that uses google search to get latest information about
current events, weather, or business hours.',
    instruction='Use google search to answer user questions about real-time, logistical
information.',
    tools=[google_search],
)

```

建立檔案後，請更新 `agent.py` 檔案，如下所示。

更新 `agent.py` 檔案

複製這個程式碼區塊，並取代 `agent.py` 檔案的現有內容：

```

from google.adk.agents import Agent
from google.adk.tools import FunctionTool
from google.adk.tools.agent_tool import AgentTool

from .custom_functions import get_fx_rate
from .custom_agents import google_search_agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='A helpful assistant for user questions.',
    tools=[
        FunctionTool(get_fx_rate),
        AgentTool(agent=google_search_agent),
    ]
)

```

我們來分析程式碼中強大的新模式：

- **全新專業代理**：我們定義了全新的代理 `google_search_agent`。請注意其具體說明，以及唯一工具 `google_search`。這是搜尋專家。
- `agent_tool.AgentTool`：這是 ADK 的特殊包裝函式。這會擷取整個代理 (我們的 `google_search_agent`)，並封裝成標準工具的外觀和行為。
- **更智慧的 `** root_agent **`**：`root_agent` 現在有新工具：`AgentTool(agent=google_search_agent)`。雖然不會搜尋網路，但知道自己有工具可以委派搜尋工作。

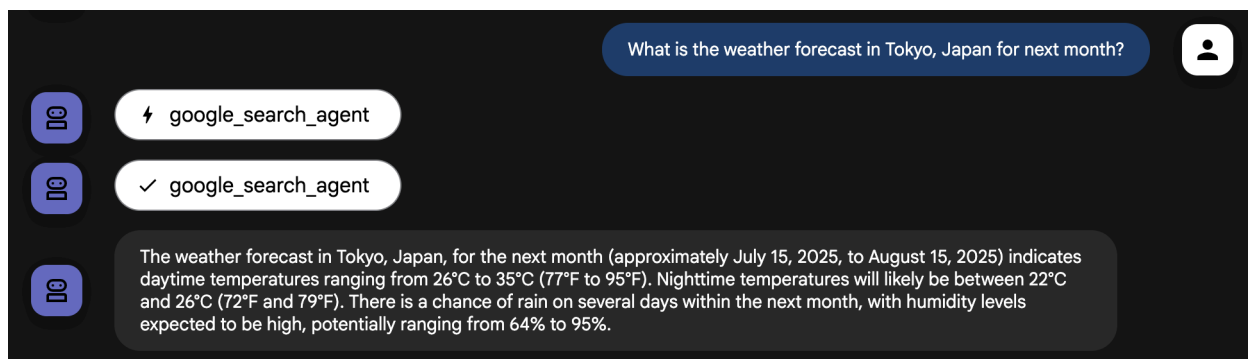
請注意，`root_agent` 中已沒有指令欄位。現在，指令會由可用的工具隱含定義。

`root_agent` 已成為自動調度管理工具或路由器，主要工作是瞭解使用者的要求，並將要求傳遞給正確的工具，也就是 `get_fx_rate` 函式或 `google_search_agent`。這種去中心化設計是建構複雜且易於維護的代理系統的關鍵。

現在，請在終端機中輸入下列指令，啟動執行個體：

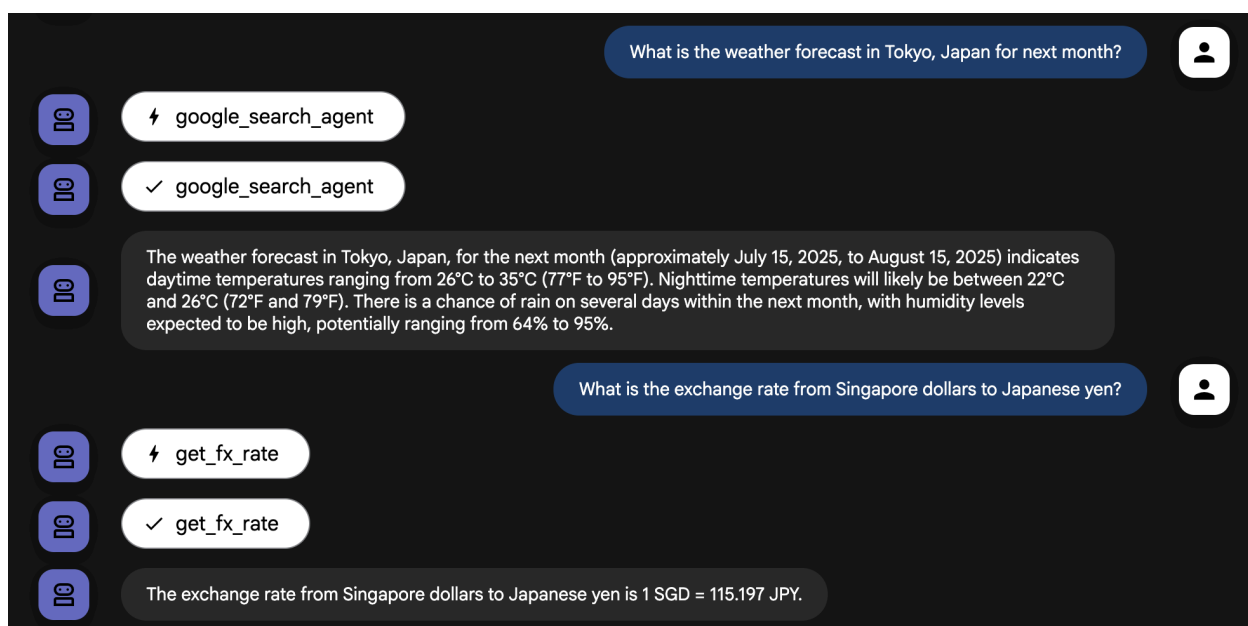
```
adk web --allow_origins "regex:https://.*\.cloudshell\.dev"
```

代理執行後，請再次向代理提出這個問題：「What is the weather forecast in Tokyo, Japan for next month?」（日本東京下個月的天氣預報為何？）



代理現在會使用 `google_search_agent` 取得最新資訊

你也可以詢問目前的兌換問題。現在服務專員應該能針對相應問題使用正確工具。



請隨意向代理提出需要即時資訊的問題，並觀察代理如何使用可用的工具處理查詢。

6. 善用 LangChain 的維基百科工具

我們的代理程式即將成為出色的旅遊助理。你可以使用 `get_fx_rate` 工具處理匯兌事宜，並使用 `google_search_agent` 工具管理物流。但一趟美好的旅程不只是安排行程，更要深入瞭解目的地的文化和歷史。

雖然 `google_search_agent` 可以找出文化和歷史事實，但來自維基百科等專門來源的資訊通常更結構化且可靠。

所幸 ADK 的設計具備高度擴充性，可完美整合其他 AI 代理框架 (例如 CrewAI 和 LangChain) 的工具。這項互通性至關重要，因為可縮短開發時間，並重複使用現有工具。針對這個用途，我們會運用 LangChain 的維基百科工具。

首先，停止執行中的代理程式程序 (**Ctrl + C**)，然後在終端機中輸入下列指令，將其他程式庫安裝至目前的 Python 虛擬環境。

```
uv pip install langchain-community wikipedia
```

建立 `third_party_tools.py` 檔案

注意：請先確認您位於正確的目錄 (`ai-agents-adk`)，再繼續操作。在終端機中執行 `pwd` (列印工作目錄) 指令，確認目前位置。

下列指令會在 `personal_assistant` 資料夾中建立名為 `third_party_tools.py` 的 Python 檔案，並在 Cloud Editor 中開啟該檔案：

```
cloudshell edit personal_assistant/third_party_tools.py
```

現在的資料夾結構應如下所示：

```
ai-agents-adk/  
├── personal_assistant/  
│   ├── .env  
│   ├── __init__.py  
│   ├── agent.py  
│   ├── custom_functions.py  
│   ├── custom_agents.py  
│   └── third_party_tools.py
```

這個檔案會包含 LangChain Wikipedia 工具的實作內容。將下列程式碼複製到 `third_party_tools.py` 檔案中：

```

from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper

# Configure the Wikipedia LangChain tool to act as our cultural guide
langchain_wikipedia_tool = WikipediaQueryRun(
    api_wrapper=WikipediaAPIWrapper(top_k_results=1, doc_content_chars_max=3000)
)

# Give the tool a more specific description for our agent
langchain_wikipedia_tool.description = (
    "Provides deep historical and cultural information on landmarks, concepts, and places."
    "Use this for 'tell me about' or 'what is the history of' type questions."
)

```

更新 `agent.py` 檔案

現在，請使用下列內容更新 `agent.py` 檔案：

```

from google.adk.agents import Agent
from google.adk.tools import FunctionTool
from google.adk.tools.agent_tool import AgentTool
from google.adk.tools.langchain_tool import LangchainTool

from .custom_functions import get_fx_rate
from .custom_agents import google_search_agent
from .third_party_tools import langchain_wikipedia_tool

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='A helpful assistant for user questions.',
    tools=[
        FunctionTool(get_fx_rate),
        AgentTool(agent=google_search_agent),
        LangchainTool(langchain_wikipedia_tool),
    ]
)

```

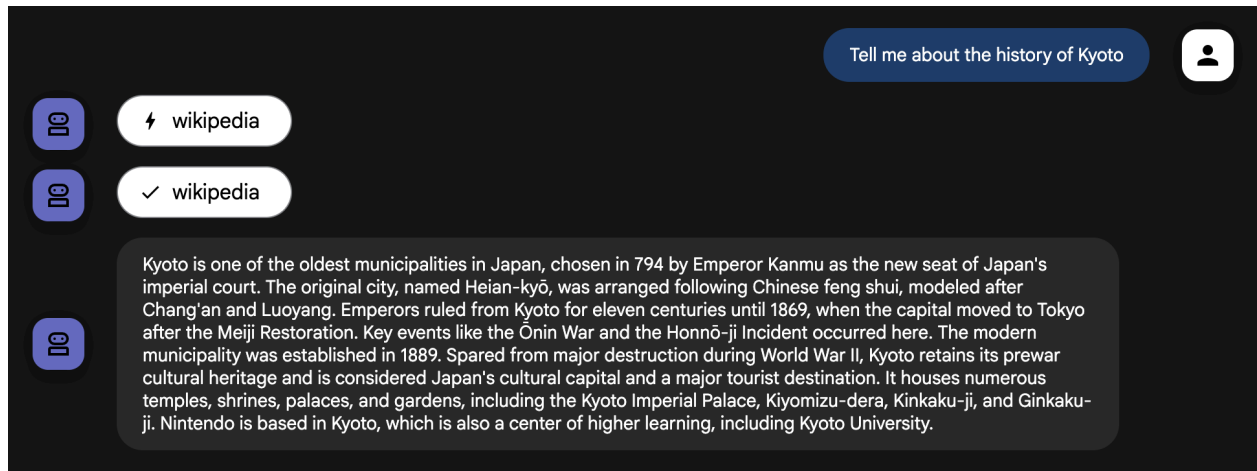
現在，請在終端機中輸入下列指令，啟動執行個體：

```

adk web --allow_origins "regex:https://.*\.cloudshell\.dev"

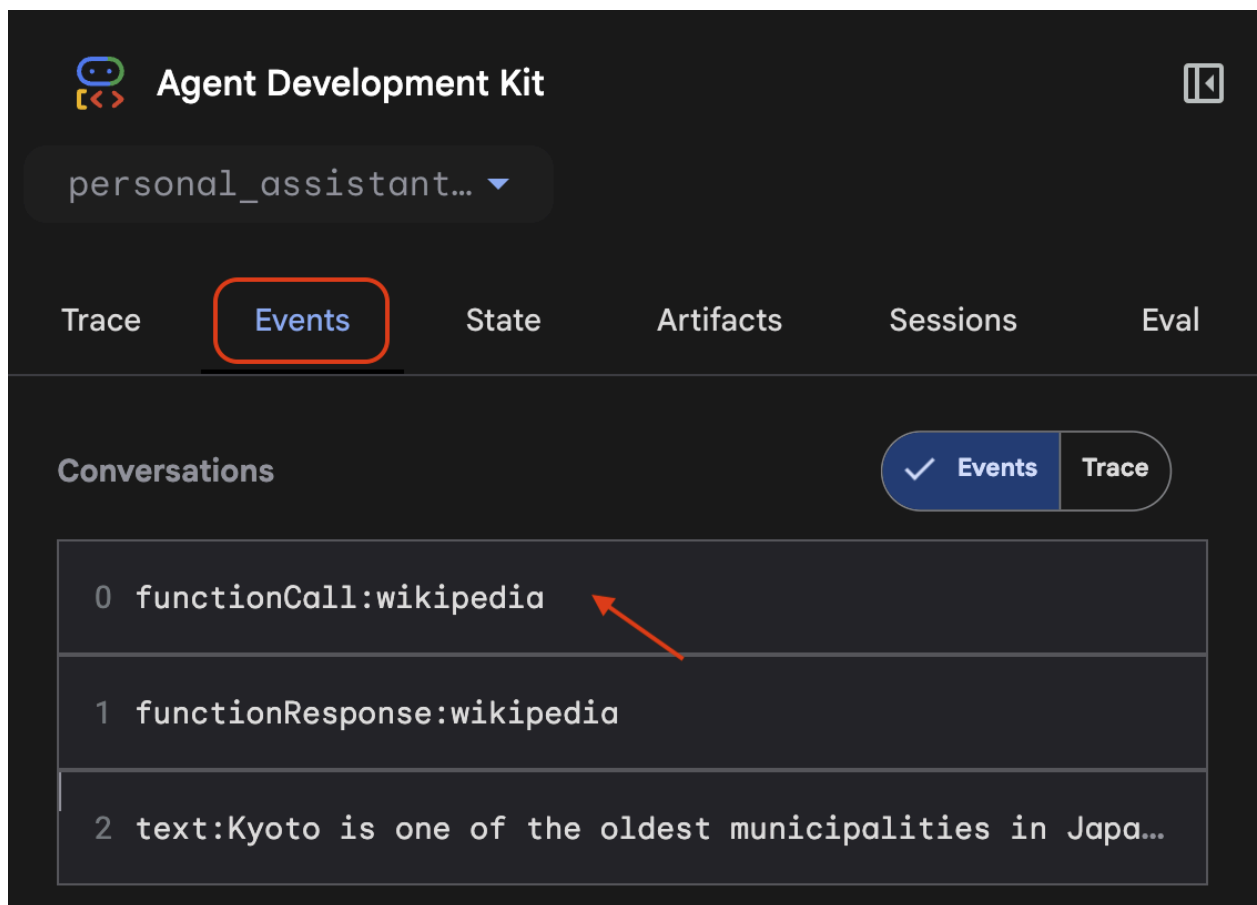
```

代理準備就緒後，向代理提出這個問題：「**Tell me about the history of Kyoto**」(告訴我京都的歷史)。

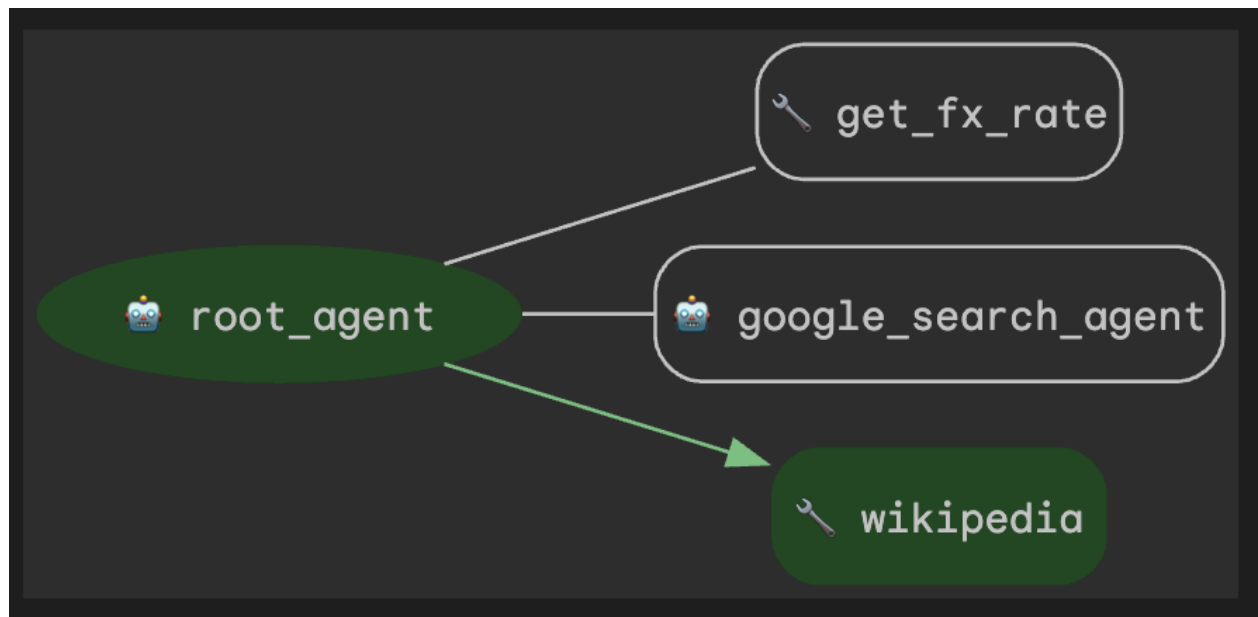


代理程式會正確將此視為歷史查詢，並使用新的 **Wikipedia** 工具。整合第三方工具並賦予特定角色後，智慧助理的智慧程度大幅提升，在規劃旅遊行程時更加實用。

如要查看代理程式做出這項選擇的確切方式，可以使用 `adk web` UI 中的事件檢查器。按一下「事件」分頁標籤，然後按一下最近的 `functionCall` 事件。



檢查器會顯示所有可用工具的清單，並醒目顯示代理程式執行的工具的 `tool_code`。



7. 清除 (選用)

注意：如果您打算繼續進行本系列課程，**就不需要執行這個步驟**。如果你是與老師一起參加這個程式碼研究室，老師會提供進一步的操作說明。

由於本程式碼研究室不涉及任何長時間執行的產品，因此只要在終端機中按下 **Ctrl + C** 鍵，停止作用中的代理程式工作階段 (例如終端機中的 `adk web` 執行個體) 即可。

刪除代理程式專案資料夾和檔案

如要從 Cloud Shell 殼層環境中移除程式碼，請使用下列指令：

```
cd ~  
rm -rf ai-agents-adk
```

停用 Vertex AI API

如要停用先前啟用的 Vertex AI API，請執行下列指令：

```
gcloud services disable aiplatform.googleapis.com
```

關閉整個 Google Cloud 專案

如要完全關閉 Google Cloud 專案，請參閱[官方指南](#)，瞭解詳細操作說明。

8. 結論

恭喜！您已成功為個人助理代理程式提供自訂函式和即時 Google 搜尋存取權。請參閱這份官方文件，瞭解如何[搭配使用工具與 Google ADK](#)。

更重要的是，您已學會建構功能強大代理的基礎架構模式：使用專門代理做為工具。建立專屬 `google_search_agent` 並提供給 `root_agent`，您已完成第一步，從建構單一代理邁向協調簡單但功能強大的多代理系統。

您現在已做好準備，可參加本系列下一個程式碼研究室 (即將推出)，深入瞭解如何協調多個代理程式和工作流程。到時見！
